MIVA OPEN UNIVERSITY

Department of Software Engineering

Course Code: SEN 306 — Software Construction
Assessment Type: Tutor-Marked Assessment (Individual Work)
Student Name: Christian Chijioke Nnaji | **Reg No:** 2024/B/SENG/0246 | **ID:** 30037606
Date: October 18, 2026

PROJECT REPORT: CUSTOMER ORDER & SUPPORT MANAGEMENT SYSTEM

1. Executive Summary & Task Overview

The goal of this project was to design and construct a modular, real-life Customer Order Processing and Support Management application demonstrating the practical integration of three core data structures: a **Queue**, a **Singly Linked List**, and a **Stack**. To adhere to strict software construction standards, no built-in Python collection array helper methods (such as native list extensions) were used for data storage or tracking configurations. Every single layer was written from scratch using a node-based pointer connection structure.

The codebase is split into clean modules (queue_module.py, linkedlist.py, stack.py, and main.py) to fulfill professional design abstractions, decouple software logic, and enforce high maintainability.

Remote Assignment Delivery Links:

• GitHub Source Code Link: https://github.com/Access2Christian/customer-order-management-system.git
• Google Drive Institutional Cloud Share: https://drive.google.com/drive/folders/1pSo8X8BwN5-3MMcPNWQij9KXC6ssh8vG?usp=sharing

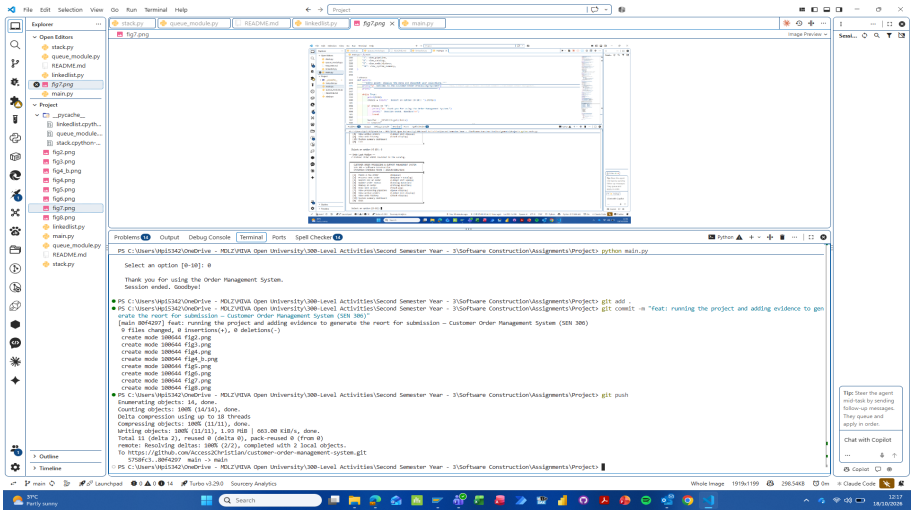


Figure 1: Version control deployment pipeline verification sync (Git Push to Remote Main Branch)

2. Functional Mapping of Data Structures

• **Queue (FIFO):** Serves as the *Inbound Order Intake Pipeline*. Checkout events are appended using enqueue() at the rear boundary and extracted via dequeue() from the front, ensuring customer orders are processed in strict chronological arrival sequence.

• **Singly Linked List:** Serves as the persistent *Active Orders Catalog Database*. It navigates reference pointer structures linearly, allowing custom runtime logic to look up records or handle target link extractions securely without fragmenting data chains.

• **Stack (LIFO):** Serves as the atomic *Undo Transaction Ledger*. Every mutation operation (insertions, flag updates, deletions) pushes state snapshots onto the top-level pointer. Popping an item from the stack rolls back variations safely.

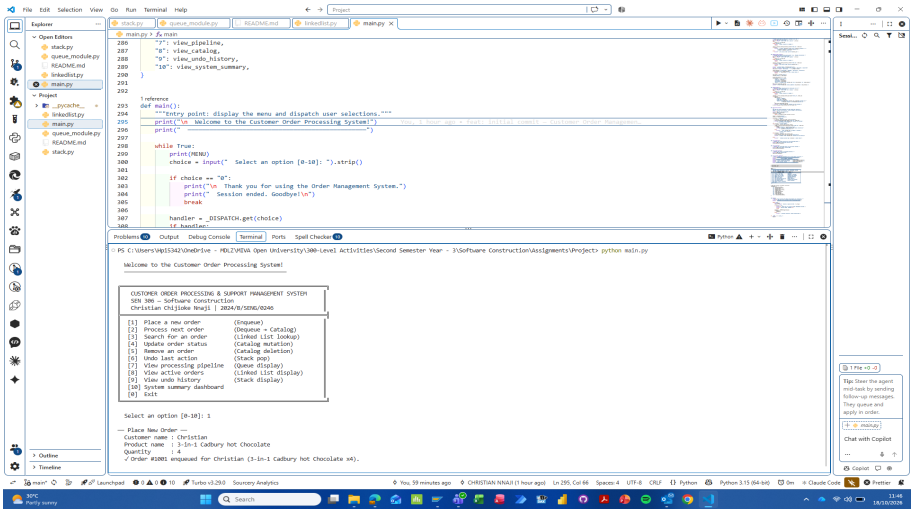


Figure 2: Inbound order intake pipeline processing simulation (Option 1: Enqueue)

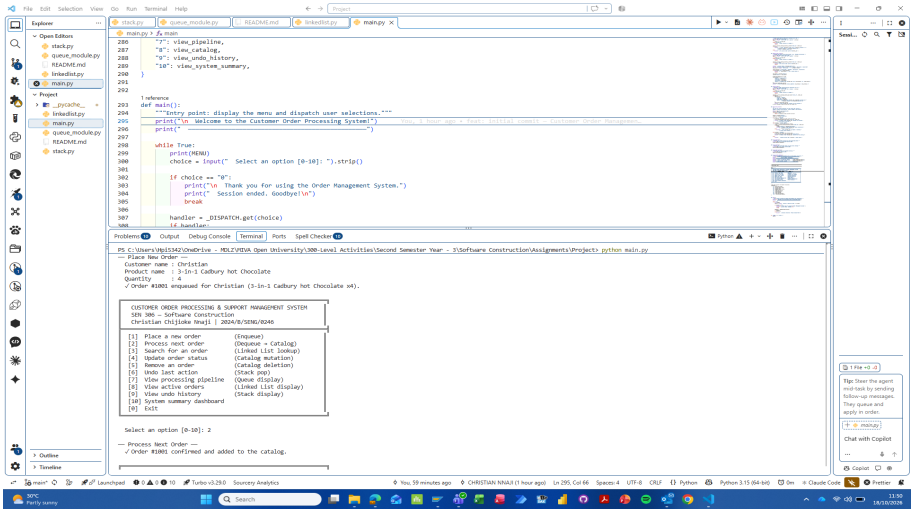


Figure 3: Pipeline verification tracking routine (Option 2: Dequeue to Database Catalog)

3. Application of Software Construction Principles

• **Modular Design:** Architecture decouples data containment components cleanly away from runtime presentation logic.

• **High Cohesion:** Classes implement single-purpose operational tracking methods (e.g., node reference shuffling rules).

• **Low Coupling:** Submodules use native Python parameter models with no interdependencies or internal cross-imports.

• **Defensive Validation:** Robust evaluation loops intercept non-numeric amounts, missing fields, or duplicate identifiers.

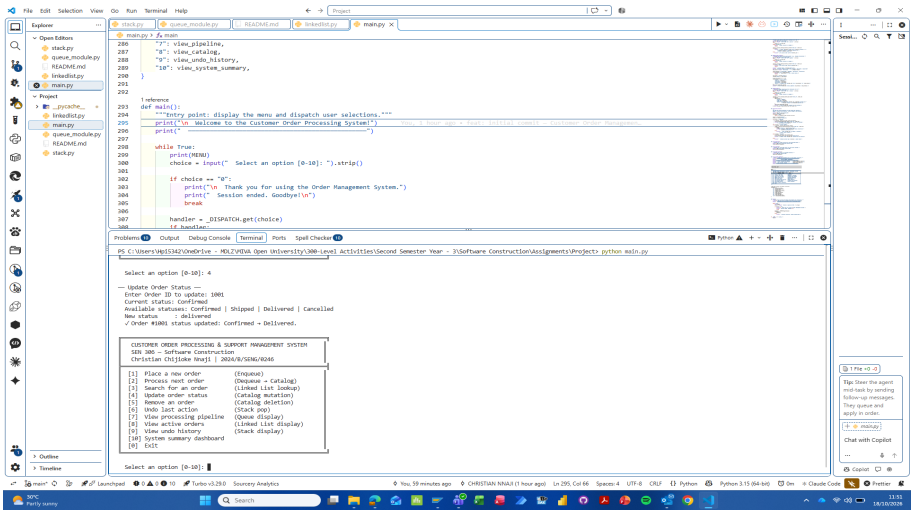


Figure 4: Data field entry adjustment tracking initialization (Option 4: Update order status flags)

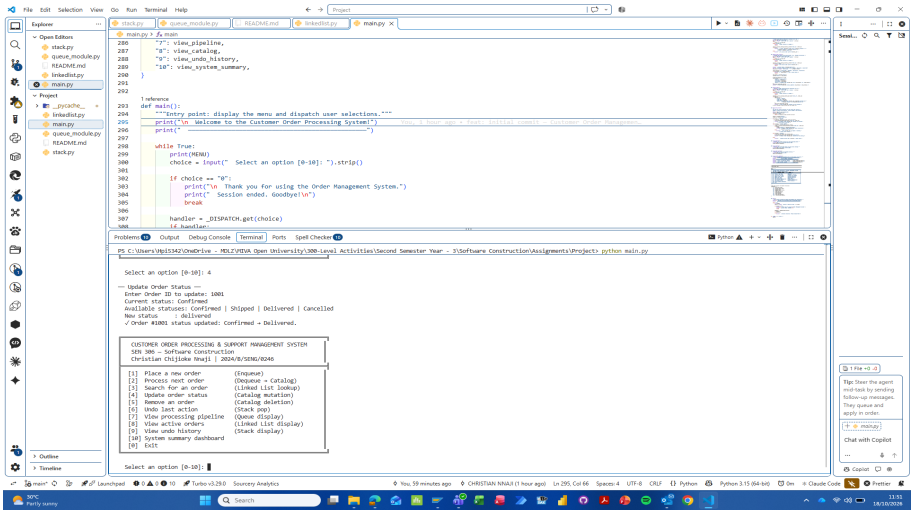


Figure 4b: Data mutation confirmation — Order status changed successfully to Delivered

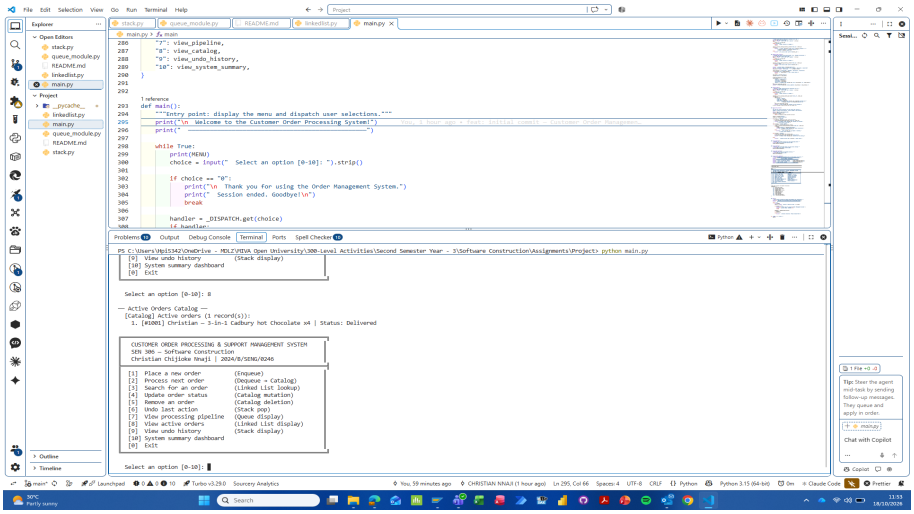


Figure 5: Active memory list database evaluation layer (Option 8: Active orders catalog look)

4. Operational Test Execution Evidence

Empirical terminal screenshots documenting state removal mutations and stack recovery transformations:

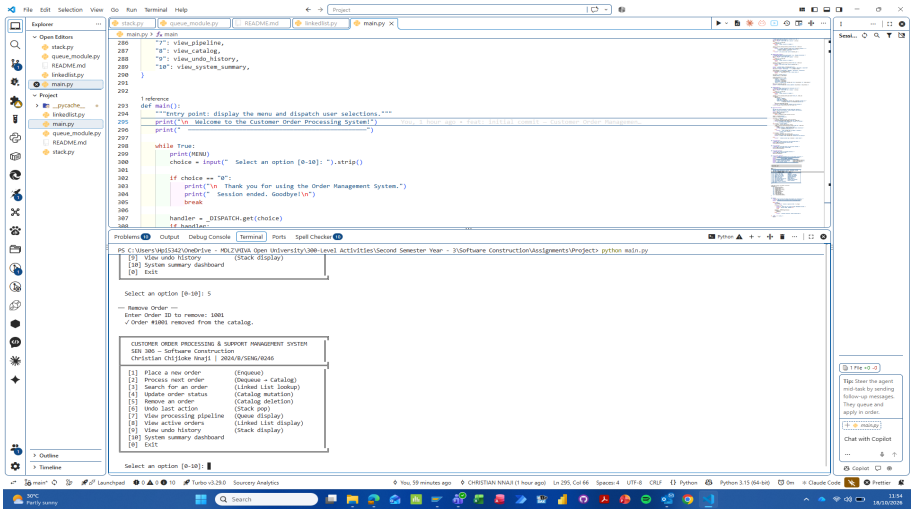


Figure 6: Destructive catalog removal tracking operation (Option 5: Remove order)

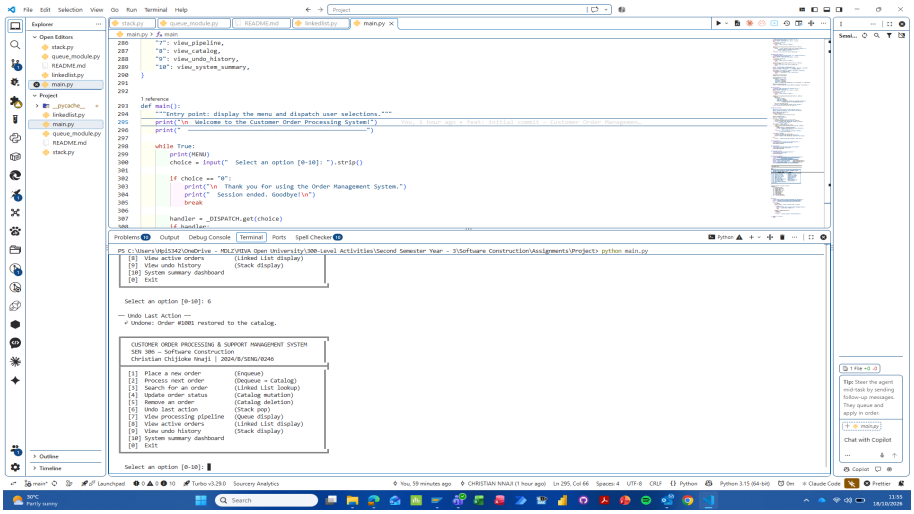


Figure 7: System rollback execution state analysis (Option 6: Undo last action)

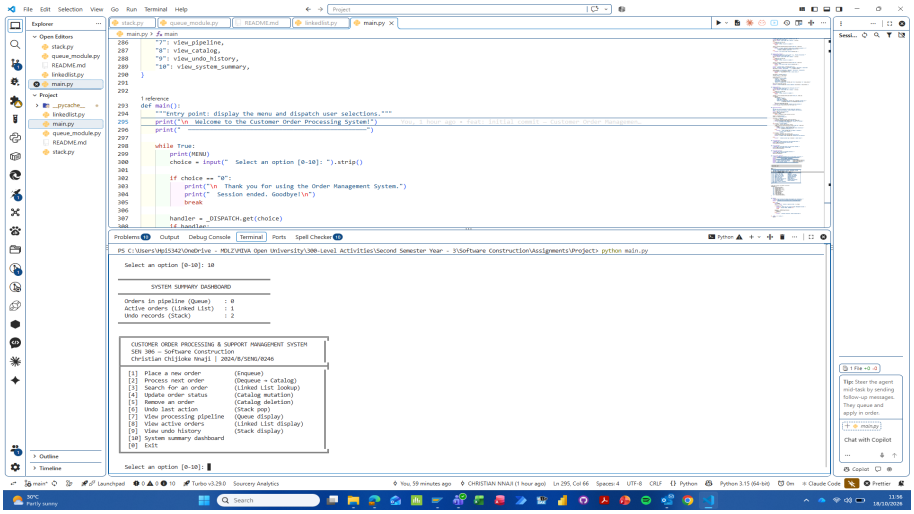


Figure 8: Quantitative operational metrics administrative overview dashboard (Option 10)